

HUBBI · Desafio Técnico Backend

Consultor de IA para Marketplace de Autopeças

Contexto

Você foi selecionado(a) para o desafio técnico da Hubbi. O objetivo é desenvolver uma API RESTful em Django que vai além do CRUD tradicional: o sistema deve ser capaz de compreender linguagem natural e sugerir peças automotivas relevantes com o auxílio de Inteligência Artificial.

O cenário simula um marketplace real de autopeças onde diferentes fornecedores cadastram seus produtos com nomes distintos para a mesma peça. Por exemplo, 'Filtro de Óleo', 'Filtro do Motor' e 'Elemento Filtrante de Óleo' são, na prática, o mesmo produto. O consultor de IA deve ser capaz de entender isso ao responder perguntas dos usuários.

Material Fornecido

Junto com este enunciado você receberá 5 arquivos CSV, cada um representando o catálogo de um fornecedor diferente. As peças se repetem entre os catálogos, porém com nomes distintos (sinônimos reais usados no mercado automotivo).

Estrutura dos arquivos CSV

Cada arquivo contém as colunas abaixo:

- nome — nome da peça conforme o fornecedor
- descricao — descrição técnica do produto
- preco — valor unitário em reais
- quantidade_inicial — estoque inicial

Exemplo de nome para a mesma peça física em catálogos diferentes:

- Fornecedor A: "Pastilha de Freio Dianteira"
- Fornecedor B: "Plaqueta de Freio Dianteiro"
- Fornecedor C: "Lona de Freio Dianteiro"

Requisitos Funcionais

1. Marketplace de Autopeças

- Listar peças disponíveis no estoque.
- Visualizar detalhes de uma peça (nome, descrição, preço, quantidade disponível).
- Somente usuários admin podem cadastrar, editar ou remover peças.

2. Popular Peças via Planilha (Assíncrono)

- Endpoint para upload de arquivo .csv com as colunas descritas acima.
- A importação deve ser processada de forma assíncrona via Celery.
- Somente usuários admin podem utilizar essa funcionalidade.

3. Autenticação JWT

- Todos os endpoints exigem autenticação via JWT.
- Permissões diferenciadas: usuários comuns visualizam; admins gerenciam.

4. Consultor de IA ★ Requisito Principal

O que deve fazer

Criar um endpoint POST `/api/consultor/` que receba uma mensagem em texto livre e retorne sugestões de peças do estoque usando Inteligência Artificial.

- A IA deve interpretar a intenção do usuário (ex.: "meu carro está fazendo barulho na roda") e sugerir as peças mais adequadas.
- O consultor deve recuperar as peças do banco de dados e enviá-las como contexto para o modelo de linguagem.
- A resposta deve conter as peças sugeridas com nome, descrição, preço e quantidade disponível.
- A IA deve lidar com sinônimos: "filtro do motor", "filtro de óleo" e "elemento filtrante" são a mesma peça e devem ser tratadas como tal.

6. Endpoint de Integração Externa ★ Plus

Atualização de estoque via API Key

Criar um endpoint seguro que permita que sistemas externos (ERPs, WMS, etc.) atualizem o estoque das peças sem necessidade de login JWT.

- A autenticação deve ser feita por API Key no header da requisição.
- Deve suportar atualização em lote (múltiplas peças em uma única chamada).
- A API Key deve ser configurável via variável de ambiente.
- Respostas claras para tentativas de acesso não autorizado (401) e peças não encontradas (404).

7. Testes Unitários

- Endpoints de listagem, cadastro, edição e remoção de peças.
- Tarefa de reposição de estoque.
- Endpoint do consultor de IA (pode usar mock da chamada à API).
- Endpoint de integração externa.

Stack Obrigatória

Tecnologia	Requisito
Framework	Django + Django REST Framework
Banco de Dados	PostgreSQL
Tarefas Assíncronas	Celery + Redis
Autenticação	JWT (djanoorestframework-simplejwt)
Testes	pytest ou unittest
IA / LLM	Modelos gratuitos do Openrouter, Gemini ou Groq

Uso de LLM

Existem opções gratuitas de uso de IA:

- OpenRouter tem modelos gratuitos
- Gemini tem uma chave que dá pra usar free
- Groq tem modelos gratuitos para utilizar

Entrega

1. Disponibilize o código em repositório público (GitHub ou GitLab).
2. O repositório deve conter um arquivo README.md com instruções para:
 - Configurar e executar o ambiente local.
 - Configurar as variáveis de ambiente (API Key da IA, banco, Redis etc.).
 - Executar as tarefas assíncronas e o cronjob.
 - Rodar os testes.
3. De preferência, inclua arquivos Docker (docker-compose.yml) para facilitar a execução.
4. Inclua os 5 arquivos CSV fornecidos na pasta do projeto (ex.: catalogs/).

Critérios de Avaliação

O que será avaliado

- Qualidade e organização do código.

- Cobertura e qualidade dos testes.
- Funcionamento correto do Consultor de IA e tratamento de sinônimos.
- Segurança do endpoint de integração externa.
- Clareza do README e facilidade para rodar o projeto.
- Boas práticas REST (status codes, serialização, tratamento de erros).
- Uso adequado de Celery para tarefas assíncronas e cronjob.

IMPORTANTE: O principal fazer é até onde você é capaz de ir utilizando TUDO que está a sua disposição. Nenhuma estratégia é proibida contanto que você faça algo com qualidade e que saiba o que foi feito.

Dúvidas? Entre em contato por meio de ti@hubbi.app.
Estamos à disposição para esclarecer qualquer questão. Boa sorte!